

# TESTE TÉCNICO

## ENGENHEIRO DE DADOS

### OBJETIVO DO TESTE

Avaliar a capacidade do candidato em construir pipelines de dados robustos, estruturar um Data Lake no BigQuery (GCP), realizar ETL/ELT de múltiplos formatos de dados e implementar boas práticas de engenharia de dados para suportar análises e automações na PSA.

---

### O QUE SERÁ FORNECIDO

Você receberá arquivos simulados em diferentes formatos, representando fontes de dados reais da PSA:

#### Arquivo 1 - dados\_clientes.csv

- Informações cadastrais de clientes da PSA
- Campos: id\_cliente, razao\_social, cnpj, porte\_empresa, setor, cidade, estado, data\_cadastro

#### Arquivo 2 - notas\_fiscais.xml

- Notas fiscais em formato XML (simulando NFe)
- Dados: número\_nota, data\_emissao, valor\_total, impostos\_detalhados, cliente\_id, itens

#### Arquivo 3 - analises\_tributarias.txt

- Registros de análises em formato texto delimitado
- Dados: id\_analise, cliente\_id, tipo\_tributo, periodo\_analise, valor\_identificado, observacoes

#### Arquivo 4 - tarefas\_projetos.json

- Tarefas e projetos em formato JSON
- Dados: id\_projeto, nome\_projeto, cliente\_id, responsavel, datas, status, horas\_trabalhadas

#### Arquivo 5 - logs\_sistema.html

- Logs de sistema em tabela HTML
- Dados: timestamp, usuario, acao, entidade\_afetada, resultado

# ENTREGAS OBRIGATÓRIAS

## 1. PIPELINE DE INGESTÃO DE DADOS (40 pontos)

**Requisito:** Criar scripts Python que realizem a ingestão dos 5 arquivos fornecidos para o BigQuery.

**O que esperamos:**

- **Script Python funcional** que:
  - Leia cada arquivo no formato original (CSV, XML, TXT, JSON, HTML)
  - Realize parsing e transformações necessárias
  - Carregue os dados no BigQuery em tabelas estruturadas
- **Tratamento de erros** e validações básicas
- **Logging** das operações realizadas
- **Código limpo e comentado**

**Tecnologias sugeridas:**

- Python 3.8+
- Bibliotecas: pandas, google-cloud-bigquery, xmltodict, beautifulsoup4, json
- BigQuery (pode usar sandbox gratuito do GCP)

**Formato de entrega:**

- Arquivo `pipeline_ingestao.py` ou notebook `.ipynb`
  - Arquivo `requirements.txt` com dependências
- 

## 2. ARQUITETURA DE CAMADAS NO DATA LAKE (30 pontos)

**Requisito:** Estruturar os dados em camadas Bronze, Silver e Gold no BigQuery.

**O que esperamos:**

**CAMADA BRONZE (Raw Data):**

- Dados brutos como foram ingeridos
- Nomeação: `psa_raw.clientes`, `psa_raw.notas_fiscais`, etc.
- Mínima transformação, mantendo formato original

**CAMADA SILVER (Curated Data):**

- Dados limpos e padronizados
- Tipos de dados corretos
- Datas em formato ISO
- Nomeação: `psa_curated.clientes`, `psa_curated.notas_fiscais`, etc.
- Remoção de duplicatas

- Tratamento de nulos
- Padronização de campos (ex: CNPJ sem formatação, valores numéricos)

#### **CAMADA GOLD (Business Data):**

- Dados agregados e prontos para consumo
- Pelo menos 2 visões analíticas, sugestões:
  - `psa_analytics.resumo_clientes_tributos` (junção cliente + análises + valores)
  - `psa_analytics.performance_projetos` (junção projetos + tarefas + horas)
- Queries otimizadas com particionamento (se aplicável)

#### **Formato de entrega:**

- Script SQL com DDL das tabelas e queries de transformação
  - Arquivo `arquitetura_camadas.sql`
  - Diagrama simples da arquitetura (pode ser em ferramenta online como draw.io, Lucidchart ou até no Google Slides)
- 

### **3. OTIMIZAÇÃO E PERFORMANCE (15 pontos)**

**Requisito:** Demonstrar conhecimento em otimização de queries e boas práticas.

#### **O que esperamos:**

- **Pelo menos 2 exemplos** de otimização aplicadas:
  - Uso de particionamento por data
  - Clustering de tabelas
  - Desnormalização estratégica
  - Índices ou filtros eficientes
- **Justificativa escrita** de cada escolha
- **Query complexa otimizada** que:
  - Una pelo menos 3 tabelas
  - Tenha agregações
  - Seja performática (explique as decisões)

#### **Formato de entrega:**

- Documento `otimizacoes.md` explicando as escolhas
  - Arquivo `queries_otimizadas.sql` com exemplos comentados
- 

### **4. QUALIDADE E GOVERNANÇA DE DADOS (10 pontos)**

**Requisito:** Implementar checagens básicas de qualidade de dados.

### O que esperamos:

- **Script de Data Quality** que valide:
  - Campos obrigatórios não nulos
  - Formatos corretos (CNPJ, datas, etc.)
  - Valores dentro de ranges esperados
  - Duplicatas
- **Documentação de metadados:**
  - Dicionário de dados (pelo menos para camada Gold)
  - Descrição de cada tabela e campo
  - Tipos de dados e restrições

### Formato de entrega:

- Arquivo `data_quality.py` ou `data_quality.sql`
  - Arquivo `dicionario_dados.md` ou `.xlsx`
- 

## 5. DOCUMENTAÇÃO TÉCNICA (5 pontos)

**Requisito:** Documentar todo o processo e decisões técnicas.

### O que esperamos:

- **README.md** completo contendo:
    - Visão geral da solução
    - Como executar os scripts
    - Dependências e pré-requisitos
    - Estrutura de pastas
    - Próximos passos e melhorias sugeridas
  - **Arquitetura visual** (diagrama simples)
  - **Decisões técnicas** justificadas
- 

## INSTRUÇÕES DE EXECUÇÃO

### Ambiente:

- Você pode usar o **BigQuery Sandbox** (gratuito, sem necessidade de cartão de crédito)
- Crie um projeto chamado `psa-data-test`

## Estrutura de entrega:

None

```
teste-engenheiro-dados/  
|  
├── scripts/  
│   ├── pipeline_ingestao.py  
│   ├── data_quality.py  
│   └── requirements.txt  
|  
├── sql/  
│   ├── arquitetura_camadas.sql  
│   └── queries_otimizadas.sql  
|  
├── docs/  
│   ├── README.md  
│   ├── dicionario_dados.md  
│   ├── otimizacoes.md  
│   └── diagrama_arquitetura.png (ou .pdf)  
|  
└── dados/ (os arquivos fornecidos)  
    ├── dados_clientes.csv  
    ├── notas_fiscais.xml  
    ├── analises_tributarias.txt  
    ├── tarefas_projetos.json  
    └── logs_sistema.html
```

## Como enviar:

1. **Repositório GitHub** (preferencial):
  - Crie um repositório público com todo o código
  - Envie o link no e-mail de resposta
2. **Ou arquivo compactado (.zip):**
  - Compacte toda a estrutura
  - Envie via Google Drive, Dropbox ou similar
  - Compartilhe o link no e-mail de resposta
3. **Acesso ao BigQuery** (obrigatório):
  - Compartilhe acesso de visualização do projeto BigQuery
  - Tutorial: adicionar `testador-psa@example.com` como viewer
  - Ou: exporte queries e envie screenshots das tabelas criadas

## CRITÉRIOS DE AVALIAÇÃO

| Critério               | Peso | O que avaliamos                                                                                                                 |
|------------------------|------|---------------------------------------------------------------------------------------------------------------------------------|
| Pipeline de Ingestão   | 40%  | • Código funcional e limpo<br>• Tratamento de erros<br>• Parsing correto de múltiplos formatos<br>• Boas práticas Python        |
| Arquitetura de Camadas | 30%  | • Estrutura Bronze/Silver/Gold bem definida<br>• Transformações adequadas<br>• Modelagem dimensional<br>• Padronização de dados |
| Otimização             | 15%  | • Técnicas de otimização aplicadas<br>• Justificativas claras<br>• Queries eficientes<br>• Conhecimento de boas práticas        |
| Qualidade de Dados     | 10%  | • Validações implementadas<br>• Documentação de metadados<br>• Dicionário de dados completo                                     |
| Documentação           | 5%   | • Clareza na explicação<br>• README completo<br>• Decisões justificadas<br>• Fácil reprodução                                   |

**Pontuação mínima para aprovação:** 70 pontos

---

## PRAZO DE ENTREGA

**Tempo:** 5 dias corridos a partir do recebimento deste teste

**Data de início:** [Data de envio do e-mail]

**Data limite:** [Data + 5 dias] até 23h59

---

## DICAS IMPORTANTES

**Comece pelos dados:**

- Explore os arquivos fornecidos antes de começar a codificar
- Entenda a estrutura e relações entre as bases

**Organize seu tempo:**

- Dia 1-2: Ingestão e camada Bronze
- Dia 3-4: Camadas Silver e Gold
- Dia 5-6: Otimizações e qualidade
- Dia 7: Documentação e revisão

**Foque em funcionamento primeiro:**

- É melhor entregar algo simples que funciona do que algo complexo pela metade
- Você pode listar melhorias futuras no README

#### **Demonstre seu conhecimento:**

- Comente seu código explicando decisões
- Use boas práticas (PEP8 para Python, por exemplo)
- Mostre que pensa em escalabilidade e manutenção

#### **Pode consultar:**

- Documentação oficial (BigQuery, Python, etc.)
- Stack Overflow para dúvidas pontuais
- Tutoriais de boas práticas

#### **Não pode:**

- Copiar soluções prontas de outros testes
- Usar ferramentas de ETL visual (Pentaho, Talend) - queremos ver código
- Usar IA para gerar todo o código (pode usar para consultas específicas)

---

## **DIFERENCIAIS (BÔNUS)**

Se quiser ir além e impressionar:

#### **☀️ Orquestração (+10 pontos):**

- Implementar um DAG simples com Apache Airflow (pode ser local)
- Ou script com agendamento e retry logic

#### **☀️ API (+10 pontos):**

- Criar uma API REST simples (Flask/FastAPI) que exponha dados da camada Gold
- Endpoint que retorna análises agregadas

#### **☀️ Testes Unitários (+5 pontos):**

- Testes para funções críticas do pipeline
- Uso de pytest ou unittest

#### **☀️ CI/CD (+5 pontos):**

- GitHub Actions ou similar configurado
- Validação automática de código

#### **☀️ Vídeo Explicativo (+5 pontos):**

- Vídeo curto (5-10min) explicando a solução

- Demonstração do funcionamento

**Máximo com bônus:** 135 pontos

---

## DÚVIDAS?

Em caso de dúvidas, você pode:

- Enviar e-mail para: [email da PSA]
- Prazo para dúvidas: até 48h antes do prazo final
- Tempo de resposta: até 24h úteis

**Importante:** Dúvidas sobre como fazer (conceitos) não serão respondidas, mas dúvidas sobre o que esperamos ou problemas técnicos (acesso, arquivos) sim!

---

## BOA SORTE!

Estamos ansiosos para ver sua solução! Este teste foi desenhado para simular desafios reais que você enfrentará na PSA.

Lembre-se: valorizamos **raciocínio lógico, código limpo, boas práticas e capacidade de documentar** tanto quanto funcionalidade.

Mostre seu melhor trabalho!